

Boosting Vector Instruction Throughput in RISC-V via a Hybrid Decoupled Architecture with VLIW-Driven Execution

Weiying Wang, Qingchen Zhai, Ruozhou Xiao, and Zhiwei Zhang

Institute of Automation, Chinese Academy of Sciences

School of Artificial Intelligence, University of Chinese Academy of Sciences

Beijing 100049, China

Email: {wangweiying2021, zhaiqingchen2021, ruozhou.xiao, zhiwei.zhang}@ia.ac.cn

Abstract—This paper addresses the instruction throughput bottleneck in traditional scalar-core-driven vector coprocessors by proposing a hybrid decoupled vector architecture driven by a Very Long Instruction Word (VLIW) scheme. The design introduces a complete workflow from task submission to synchronization: the host core configures tasks via memory-mapped CSRs, triggers execution, and is then released to achieve coarse-grained task-level parallelism; the vector coprocessor integrates hardware task scheduling, dual-buffered instruction prefetching, dynamic VLIW packing, and scalar instruction execution capabilities to support fine-grained instruction-level parallelism. Benchmarking against the Ara processor demonstrates that under short-vector scenarios (AVL=32), the proposed architecture achieves a 44% performance improvement while increasing vector unit utilization from 40.2% to 67.9%. Although the performance advantage narrows with increasing vector length, the architecture consistently outperforms traditional design in both vector unit utilization and overall performance (e.g., +3.2% perf. at AVL=512). Furthermore, it exhibits excellent area and power characteristics, comprehensively validating the design’s effectiveness.

Index Terms—RISC-V, VLIW, Hybrid decoupled architecture, Parallelism

I. INTRODUCTION

With the rapid development of data-parallel intensive applications such as deep neural networks, high-performance computing, and autonomous driving, the computational demands of these applications have grown exponentially [1], [2]. This surge exceeds the gains from traditional iterative methods, necessitating data-level parallelism to overcome performance bottlenecks [3]. The RISC-V Vector Extension (RVV) [4] offers a breakthrough solution for data-intensive computing by leveraging its open-source flexibility, efficient SIMD parallelism, and variable-length vector architecture to deliver high performance and energy efficiency.

The heterogeneous architecture combining a scalar control core with a multi-lane vector coprocessor has become a mainstream implementation paradigm for RISC-V vector processors. In this architecture, the scalar core handles irregular tasks such as program control and address calculation, while the vector coprocessor serves as an acceleration engine dedicated to data-parallel tasks. By enabling specialized division of labor

between control-intensive and computation-intensive workloads, this approach achieves an effective balance between energy efficiency and implementation complexity.

The practical effectiveness of this paradigm critically depends on the efficiency with which the scalar core drives the vector coprocessor. As demonstrated in the Ara processor study [5], significant performance degradation occurs when processing short vector lengths or problem sizes comparable to the number of vector lanes. The identified performance bottleneck does not come from the vector coprocessor itself, but rather from the scalar core’s insufficient instruction issue rate, failing to deliver adequate instruction throughput for the vector pipeline. This results in low utilization of the vector computation units that constitute the majority of hardware resources, causing the processor’s computational capability to fall substantially below peak potential, thereby undermining the high-performance and high-energy-efficiency advantages of RISC-V vector processors.

To address the aforementioned issues, existing research [6]–[9] predominantly focuses on enhancing the instruction issue capability and dynamic scheduling capacity of the scalar host core. Such approaches, however, significantly increase design complexity and introduce substantial power and area overhead. This work shifts the optimization focus from replacing or enhancing the scalar core to improving the instruction throughput of the vector coprocessor itself. To this end, we propose a VLIW-driven hybrid decoupled vector architecture. The principal contributions of this paper are as follows:

- 1) We propose a hybrid decoupled vector architecture that breaks away from the traditional tightly-coupled model dominated by a scalar core. The “decoupled” aspect refers to coarse-grained task-level parallelism between the host scalar core and the vector coprocessor: the host core packages and submits tasks before releasing resources. The “hybrid” aspect indicates that the vector coprocessor integrates scalar instruction execution capability, enabling autonomous and asynchronous processing of all instructions within a task package for fine-grained instruction-level parallelism.
- 2) We introduce a dynamic VLIW instruction packing

scheme based on RISC-V hint instructions [10]. This method achieves explicit instruction-level parallelism performance benefits similar to traditional VLIW [11], without requiring modifications to the existing RISC-V instruction set, thereby maintaining the flexibility and compatibility of the vector programming model.

- 3) We present the microarchitecture design for key components enabling high-throughput instruction and task execution, comprising: a) a hardware Task Schedule Unit supporting cycle-level responsive task queuing and initiation; b) a dual-buffered Instruction Prefetch Unit with a loop optimization mechanism to eliminate fetch overhead in critical loops; and c) a Hybrid Execution Unit capable of simultaneous scalar and vector operation execution. The co-design of these components ensures sustained high-throughput instruction supply and execution.
- 4) We define a complete operational workflow from task submission to completion, providing a practical programming model. This includes task configuration via memory-mapped CSRs [12], autonomous execution by the vector coprocessor, and lightweight synchronization based on status polling.

II. HYBRID DECOUPLED VECTOR ARCHITECTURE

This section elaborates on the implementation details of the hybrid decoupled vector architecture in Fig. 1.

A. Task Interface Unit

The Task Interface Unit (TIU) interacts with the host core through a set of custom memory-mapped CSRs, providing software with a unified and concise task control interface. As shown in Table I, these CSRs, including VTASK_ADDR, VTASK_PADDR, VTASK_START, and VTASK_STATUS, collectively define the fundamental execution environment for vector tasks.

TABLE I
CONFIGURATION CSRS FOR VECTOR TASK SCHEDULING

CSR	Address	Description
VTASK_ADDR	0x7C0	Vector task address register
VTASK_PADDR	0x7C1	Vector task descriptor address register
VTASK_START	0x7C2	Vector task start control register
VTASK_STATUS	0x7C3	Vector task status register. BUSY: task busy status; DONE: task completion flag; ERROR: task error flag

B. Task Schedule Unit

The Task Schedule Unit (TSU) acquires vector task information from the TIU and coordinates internal hardware resources to dispatch vector tasks in an orderly manner. The TSU employs a FIFO-based hardware task scheduling mechanism, which sequentially writes vector task information into a task queue according to the submission order and dispatches them in sequence to the Instruction Prefetch Unit. The TSU closely collaborates with the TIU, achieving cycle-level task trigger responses by monitoring status changes in the VTASK_START

register, and efficiently interacts with modules such as the Instruction Prefetch Unit via dedicated hardware signals.

C. Instruction Prefetch Unit

The Instruction Prefetch Unit (IPU) efficiently supplies the instruction stream to the backend of the architecture [13]. Upon receiving a task dispatched from the TSU, the IPU is immediately activated: the fetch address generation unit uses the vector task address as the initial PC value, then initiates a burst prefetch of the vector instruction sequence into the instruction buffer via a dedicated instruction fetch bus. The instruction buffer employs a dual-buffer structure, with each buffer sized at 64 bytes, sufficient to hold four 128-bit VLIW fetch packets. The VLIW Pack Unit fetches, packs, and dispatches instructions from the execution buffer, while the prefetch buffer loads subsequent instruction blocks in the background. Buffer A and Buffer B seamlessly switch roles between the prefetch buffer and execution buffer through a state machine. When processing non-contiguous instruction streams, if a miss occurs, the current prefetch is paused and a new request is initiated. Targeting the short-loop intensive characteristics of vector tasks, a specific optimization for loop bodies is implemented. The loop optimization control module automatically detects the pattern of vsetvli/vsetivli/vsetvl instructions combined with backward branches. When a vector loop fits within the 128-byte buffer capacity, its instructions are locked across both buffers, eliminating fetch overhead for subsequent iterations and saving energy.

D. VLIW Pack Unit

This architecture employs a 128-bit fetch packet for its VLIW instruction, typically containing 4 to 7 instructions. The upper 32 bits are specifically designed as a dedicated RISC-V hint instruction, whose immediate field encodes parallel control information (p-bits) [14], forming a "header word." This divides the fetch packet into multiple 16-bit instruction slots. Each p-bit in the hint instruction explicitly indicates whether the corresponding instruction slot can execute in parallel with the next slot. A p-bit of 1 enables parallel execution with the next instruction, while 0 terminates the current execute packet. A 32-bit instruction naturally occupies two consecutive slots, and its corresponding p-bits are forced to 1. An execute packet is composed of multiple parallelizable instructions defined by the hint instruction; instructions within the same execute packet can be issued in parallel within a single cycle. This design maintains RISC-V compliance while enabling scalable VLIW-style instruction-level parallelism.

During execution, the VLIW Pack Unit (VLIWPU) receives the 128-bit fetch packet from the IPU. The data dependency detection module analyzes data dependencies between instructions in real-time. Simultaneously, the parallel pack module analyzes the parallel relationships between instructions based on the "header word" in real-time: it scans the p-bit of each instruction sequentially within the fetch packet. Instructions without data dependencies and that have a parallel relationship can be merged into the same execute packet. This process

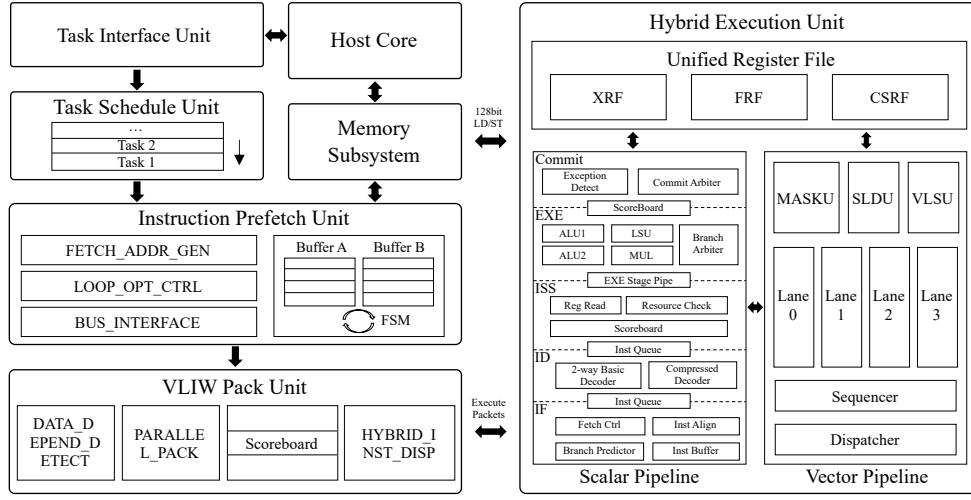


Fig. 1. Architecture Overview.

can partition the entire fetch packet into several execute packets of varying sizes. The scoreboard efficiently manages scalar-vector data dependencies across execute packets through unified register state tracking. After passing this check, the hybrid instruction dispatch module synchronously delivers the instructions from the execute packet to the scalar and vector pipelines, respectively.

E. Hybrid Execution Unit

The Hybrid Execution Unit (HEU) is the computational core of the architecture, employing a heterogeneous dual-pipeline design to support mixed vector-scalar workloads. This concurrency allows scalar instructions (e.g., for control flow and scalar calculation) to proceed in parallel with vector computations. The scalar pipeline is a streamlined five-stage, dual-issue, in-order design, equipped with two ALUs, a multiplier, an LSU, a branch predictor, among others. The vector pipeline is based on the scalable Ara project [15] and adopts a configuration with four lanes and VLEN=1024. It also integrates internal components such as a dispatcher, sequencer, VLSU, SLDU, and MASKU, providing full support for the RISC-V Vector 1.0 Extension (RVV). This full-featured vector pipeline is used for experimental validation, though a much simpler implementation is permitted in the future. Both pipelines interact independently with the Unified Register File, which facilitates the seamless exchange of data between scalar and vector operations.

III. OPERATIONAL WORKFLOW

This section outlines the operational workflow of the proposed architecture, organized into three primary phases.

A. Vector Task Configuration and Submission

At the start of the process, the host core defines a vector task descriptor in memory, which includes parameters such as input data address, output data address, and data length.

To submit the task, the core then configures the vector-task-related CSRs, including the vector task entry address and task descriptor address. With the configuration complete, the host core triggers vector task execution by setting VTASK_START and is consequently released to execute other tasks, thus realizing task-level parallelism.

B. Autonomous Vector Task Execution

When TSU detects the setting of VTASK_START, it retrieves the vector task information from TIU, sets the BUSY bit in VTASK_STATUS, and clears VTASK_START. Subsequently, IPU initiates batch prefetching of instruction sequences from memory into the instruction buffer. VLIWPU then retrieves fetch packets, analyzes runtime dependencies, and packages parallelizable instructions into execute packets for HEU dispatch. HEU autonomously executes scalar or vector instructions within the execute packet.

C. Vector Task Completion and Synchronization

Upon vector task completion, the hardware autonomously sets the DONE bit and clears BUSY in VTASK_STATUS. The vector coprocessor subsequently transitions to an idle state, awaiting the next task assignment. The host core monitors task completion by polling the DONE bit in VTASK_STATUS. Upon detecting successful execution, it utilizes the resulting data for subsequent processing and finally resets all CSRs to their initial state.

IV. EXPERIMENTAL EVALUATION AND ANALYSIS

This section presents the Performance, Power, and Area (PPA) analysis of the proposed architecture.

A. Performance

This paper conducts a performance comparison between the proposed hybrid decoupled vector architecture (H) and a traditional architecture (T) represented by the Ara vector processor. The evaluation employs typical OpenBLAS kernels

[16]—sdot, saxpy, and sgemm—implemented using hand-optimized assembly code. These kernels span application scenarios involving short to long vectors, enabling a comprehensive assessment of the architectural adaptability across varying workload dimensions.

Experimental data clearly demonstrate the significant performance advantages of the H architecture over the T architecture. As shown in Fig. 2, in short-vector scenarios (AVL=32), the H architecture exhibits the most prominent performance improvement. While the T architecture achieves only 0.65 GFLOP/s with 40.2% vector utilization, the H architecture boosts performance to 0.93 GFLOP/s with 67.9% utilization—a speedup ratio of 1.44. At medium vector length (AVL=256), the H architecture maintains a solid 6.3% performance advantage, delivering 1.22 GFLOP/s versus T’s 1.14 GFLOP/s, while achieving 76.1% vector utilization compared to T’s 68.2%. As the vector length increases to AVL=512, the performance advantage narrows to 3.2% (1.24 vs. 1.20 GFLOP/s), though the H architecture continues to demonstrate superior vector unit utilization at 76.9% versus T’s 71.3%. For matrix operations exemplified by sgemm, the H architecture consistently outperforms the traditional design, achieving 8.1–48.1% higher performance across various matrix sizes while maintaining 5.7–17.3% better utilization, demonstrating robust performance scalability in complex matrix computations.

This performance improvement stems from a triad of architectural advantages: VLIW-based parallel issuance breaking the instruction issue bottleneck, sustained prefetch supply ensuring continuous instruction flow, and enhanced hybrid execution capability with efficient scalar-vector coordination, which proves especially critical in short-vector scenarios. The performance gap narrows gradually yet remains stable with increasing AVL. This convergence aligns with expectations: longer vectors not only supply more schedulable instructions to mitigate the instruction throughput bottleneck in the T architecture but also drive computation toward memory bandwidth limits and peak performance, at which point instruction throughput ceases to be the primary constraint. Despite this, the H architecture maintains superior vector unit utilization and performance across all AVLs, thereby highlighting its overall architectural efficiency.

B. Area and Power

The H architecture demonstrates advantages in both chip area and power consumption, as measured through synthesis with Synopsys Design Compiler under TSMC 28nm HPC+ [17] and power analysis [18] using SAIF files from gate-level simulations. Experimental results in Table II show that the H architecture achieves a 4.32% reduction in chip area and an 8.75–9.97% power reduction on kernels including sdot, saxpy, and sgemm. These results suggest that a future streamlined vector pipeline would enable even greater area savings. These benefits primarily stem from the following innovations: the VLIW mechanism employs a static scheduling strategy based on RISC-V hint instructions to transform complex dynamic instruction scheduling logic into simple parallel instruction

packing, significantly streamlining the hardware structure; furthermore, the specialized design for vector kernels fully leverages their simple control flow and regular computational patterns, enabling highly efficient utilization of hardware resources; additionally, collaborative optimization techniques such as hardware-level loop optimization, dual-buffer structure, and prefetch strategies effectively reduce memory access power consumption and enhance instruction supply efficiency.

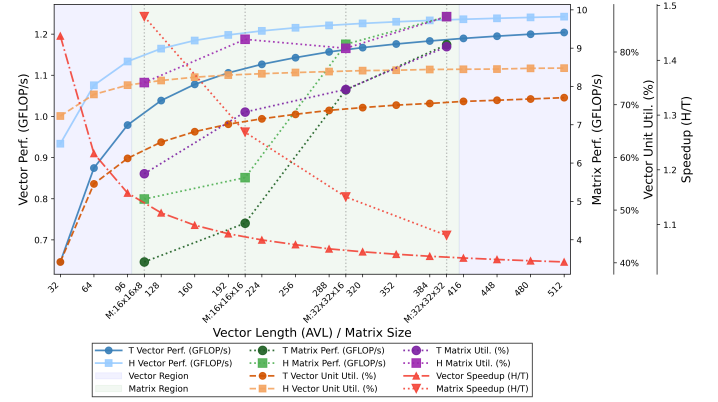


Fig. 2. Performance, Vector Unit Utilization, and Speedup (geometric mean) versus Application Vector Length (AVL) for Traditional (T) and Hybrid Decoupled (H) Architectures. The sgemm kernel is evaluated on a subset of fixed-size problems.

TABLE II
PPA METRICS: TRADITIONAL VS. HYBRID DECOUPLED ARCHITECTURE

Merit	Traditional Arch.			Hybrid Decoupled Arch.		
	sdot	saxpy	sgemm	sdot	saxpy	sgemm
Total Cycles	531	378	1846	500	355	1457
Perf. (GFLOP/s)	0.96	1.35	4.43	1.02	1.44	5.62
Vector Unit Util. (%)	90.6	51.3	68.6	97.0	59.7	82.4
Total Power (mW)	177.97	195.96	259.66	162.39	176.42	233.94
Die Area (μm^2)	4584011.12			4385981.78		

Process: TSMC 28nm HPC+ (TT/0.9V/25°C) @1GHz.

Configuration: sdot, saxpy: AVL = 256; sgemm: $16 \times 16 \times 16$.

V. CONCLUSION

This research constructs a VLIW-driven hybrid decoupled architecture that successfully breaks through the traditional instruction throughput bottleneck. It innovatively adopts a task-level decoupling paradigm, shifting from instruction-level interaction to task-level collaboration, significantly improving the system’s parallel efficiency. The dynamic VLIW packing based on RISC-V hint instructions, coordinated with autonomous hardware units, effectively exploits instruction-level parallelism while maintaining ISA compatibility. Experiments demonstrate that the architecture achieves significant performance, area, and power advantages across various vector lengths. Future work will prioritize the establishment of compiler toolchains and robust handling of memory arbitration and exceptions, while also pursuing techniques like dynamic instruction fusion, multi-core scheduling, and reconfigurable architecture to evolve the prototype into a general-purpose RISC-V accelerator paradigm.

REFERENCES

- [1] J. Sevilla, L. Heim, A. Ho, T. Besiroglu, M. Hobbhahn, and P. Villalobos, "Compute trends across three eras of machine learning," in *2022 international joint conference on neural networks (IJCNN)*. IEEE, 2022, pp. 1–8.
- [2] W.-c. Feng, K. Cameron, and T. Scogland, "The green500 list: A look back to look forward," in *Contemporary High Performance Computing*. Chapman and Hall/CRC, 2017, pp. 31–41.
- [3] J. L. Hennessy and D. A. Patterson, *Computer architecture: a quantitative approach*. Elsevier, 2011.
- [4] A. Krste and W. o. Andrew, "Working draft of the proposed risc-v v vector extension," Online, May 2023, accessed: 2025-08-20. [Online]. Available: <https://github.com/riscvarchive/riscv-v-spec>
- [5] M. Cavalcante, F. Schuiki, F. Zaruba, M. Schaffner, and L. Benini, "Ara: A 1-ghz+ scalable and energy-efficient risc-v vector processor with multiprecision floating-point support in 22-nm fd-soi," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 28, no. 2, pp. 530–543, 2019.
- [6] Y. Gao, R. Egawa, H. Takizawa, and H. Kobayashi, "An out-of-order vector processing mechanism for multimedia applications," in *Proceedings of the 9th conference on Computing Frontiers*, 2012, pp. 233–236.
- [7] R. Espasa. Implementation of an out-of-order risc-v vector unit. SemiDynamics Technology Services. [Online]. Available: <https://www.youtube.com/watch?v=WzID6kk8RN8>
- [8] R. Espasa, M. Valero, and J. E. Smith, "Out-of-order vector architectures," in *Proceedings of 30th Annual International Symposium on Microarchitecture*. IEEE, 1997, pp. 160–170.
- [9] F. Minervini, O. Palomar, O. Unsal, E. Reggiani, J. Quiroga, J. Marimon, C. Rojas, R. Figueras, A. Ruiz, A. Gonzalez *et al.*, "Vitruvius+: An area-efficient risc-v decoupled vector coprocessor for high performance computing applications," *ACM Transactions on Architecture and Code Optimization*, vol. 20, no. 2, pp. 1–25, 2023.
- [10] A. Waterman and K. Asanović, *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, RISC-V International, May 2025. [Online]. Available: <https://riscv.org/wp-content/uploads/2019/12/riscv-spec-20191213.pdf>
- [11] J. A. Fisher, "Very long instruction word architectures and the eli-512," in *Proceedings of the 10th annual international symposium on Computer architecture*, 1983, pp. 140–150.
- [12] A. Waterman and K. Asanović, *The RISC-V Instruction Set Manual: Volume II: Privileged Architecture*, RISC-V International, May 2025. [Online]. Available: <https://riscv.org/wp-content/uploads/2019/12/riscv-spec-20191213.pdf>
- [13] M. Payami, E. Azarkhish, I. Loi, and L. Benini, "A hybrid instruction prefetching mechanism for ultra low-power multicore clusters," *IEEE Embedded Systems Letters*, vol. 9, no. 4, pp. 125–128, 2017.
- [14] Texas Instruments, *TMS320C66x DSP CPU and Instruction Set Reference Guide*, Texas Instruments Incorporated, Dallas, TX, USA, November 2010. [Online]. Available: <http://www.ti.com/lit/ug/sprugh7/sprugh7.pdf>
- [15] PULP Platform, "Ara: a 64-bit Vector Unit," <https://github.com/pulp-platform/ara>, 2021, version 2.2.0.
- [16] OpenBLAS Development Team, "OpenBLAS: an optimized BLAS library," <https://github.com/OpenMathLib/OpenBLAS>, 2024, version 0.3.30.
- [17] Taiwan Semiconductor Manufacturing Company, *TCBN28HPCPLUSBWP12T40P140 TSMC N28HPC Standard Cell Library Datasheet*, March 2017.
- [18] Synopsys, Inc., *Power Compiler™ User Guide*, March 2022, version T-2022.03. [Online]. Available: <https://www.synopsys.com/>